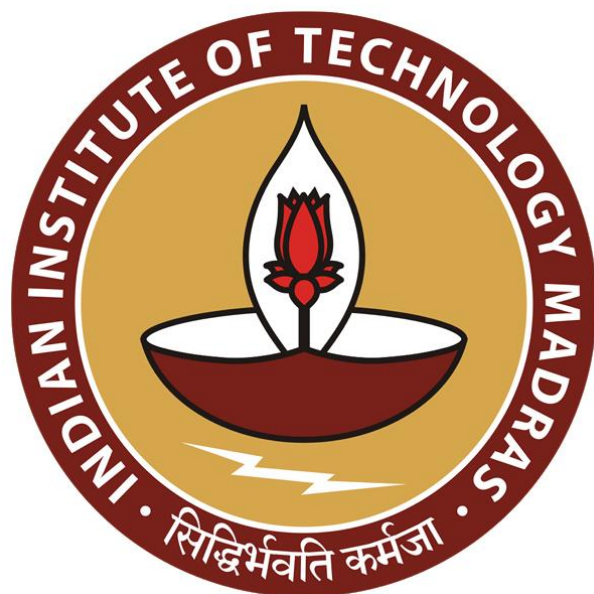




IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology Madras
Callbacks

(Refer Slide Time: 00:13)



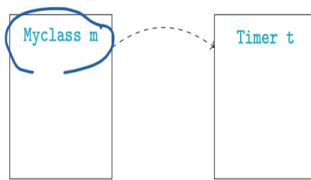
So, we saw that if we combine private class with the interface then we can provide controlled access to the state of an object. We can have some kind of regulated way in which we can interact with an object. Now, we will look at another example of what you can do with interfaces and private classes and this is called callback.

(Refer Slide Time: 00:50)

IIT Madras BSc Degree

Implementing a call-back facility

- `Myclass m` creates a `Timer t`



```
graph LR; m[Myclass m] -.-> t[Timer t]
```

Mathavan Mukund

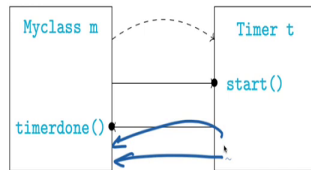
Callbacks

Programming Concepts using Java 2 / 6

IIT Madras BSc Degree

Implementing a call-back facility

- `Myclass m` creates a `Timer t`
- Start `t` to run in parallel
 - `Myclass m` continues to run
 - Will see later how to invoke parallel execution in Java!
- `Timer t` notifies `Myclass m` when the time limit expires
 - Assume `Myclass m` has a function `timerdone()`



```
graph LR; m[Myclass m] -.-> t[Timer t]; m -- start() --> t; t -- timerdone() --> m
```

Mathavan Mukund

Callbacks

Programming Concepts using Java 2 / 6

Imagine we want to have a timer as a part of our code. So we have some class that creates a `Timer t` and of course idea of timer is that imagine we are doing something in the kitchen and we set a timer, then usually what happens is that we put the cooker on the gas and then we say we want to switch it off after 10 minutes. So, we will set a timer for 10 minutes. And then we will go about doing whatever we are doing. And at the end of the 10 minutes, we expect that this timer will ring or somehow tell us that the time has elapsed and then we can get back and attend to whatever we asked, suppose, like switch off the gas or whatever.

So, we want to do a similar thing in Java or object-oriented environment. So, this class is trying to do something for which it needs some time to elapse. So, what myclass does is it creates a timer object, so timer is another class, it creates a timer object t and it starts it off. So, it starts it off. In this case, let us assume it is a simple timer with a fixed time limit. So, otherwise, you could imagine that there is an argument here which tells you how much time I want it to run for. But let us, for a moment, let us just avoid that. So, it is going to run for a fixed amount of time.

Of course, there is another complication, which is that this timer in real life is running in parallel with whatever we are doing on the side. So, we are kind of waiting to switch off the gas, but we are doing something else, we are chopping something or we have gone out and we are talking to someone on the phone or something. Now, in a Java kind of environment what you want is that this timer will run in parallel with whatever myclass is doing internally. So, that requires a different type of execution model. So, we will talk about it later how to start computation which runs in parallel, in a parallel thread as it is called.

But for the moment let us assume now that myclass is running and doing something else, and timer is running, and it is keeping time. So, it has been started at some time. And now when the time elapses, it has to come back and tell the class that it is done. So, this has to in turn come back and call a function. So, we have to assume that the class which created the timer implements some kind of a function which is callable by the timer. So, the timer knows what to do.

It knows that it should go back to the calling class, the class that created it, and call, in this case, a function called timer done. So, timer done will now have some code which myclass will execute, indicating that it is aware of the timer having elapsed and whatever needed it to do after the timer elapsed. So, this is the kind of scenario that we want to program. So, the question really is, so this is called a callback. So, when we have this kind of a function which comes back, so we create an object, we run it in parallel, and then we expect that parallel that parallel object to call us back when it is done. So, this is a very typical scenario you can imagine. And this is one concrete simple example of this.

(Refer Slide Time: 03:35)



Implementing callbacks

■ Code for Myclass

```
public class MyClass{  
  
    public void f(){  
        ..  
        Timer t =  
            new Timer(this);  
        // this object  
        // created t  
        ...  
        t.start(); // Start t  
        ...  
    }  
  
    public void timerdone(){...}  
}
```



Madhavan Mukund

Callbacks

Programming Concepts using Java 3 / 6



Implementing callbacks

■ Code for Myclass

```
public class MyClass{  
  
    public void f(){  
        Timer t =  
            new Timer(this);  
        // this object  
        // created t  
        t.start(); // Start t  
        ...  
    }  
  
    public void timerdone(){...}  
}
```



Madhavan Mukund

Callbacks

Programming Concepts using Java 3 / 6

सिद्धिर्भवति कर्मजा

Implementing callbacks

- Code for **Myclass**
- **Timer t** should know whom to notify
 - **Myclass m** passes its identity when it creates **Timer t**

```
public class Myclass{
    public void f(){
        ..
        Timer t =
            new Timer(this);
        // this object
        // created t
        ...
        t.start(); // Start t
        ...
    }

    public void timerdone(){...}
}
```



Madhavan Mukund

Callbacks

Programming Concepts using Java 3 / 6

Implementing callbacks

- Code for **Myclass**
- **Timer t** should know whom to notify
 - **Myclass m** passes its identity when it creates **Timer t**
- Code for **Timer**
 - Interface **Runnable** indicates that **Timer** can run in parallel

```
public class Myclass{
    public void f(){
        ..
        Timer t =
            new Timer(this);
        // this object
        // created t
        ...
        t.start(); // Start t
        ...
    }

    public void timerdone(){...}
}

public class Timer
    implements Runnable{
    // Timer can be
    // invoked in parallel

    private Myclass owner;

    public Timer(Myclass o){
        owner = o; // my creator
    }

    public void start(){
        ...
        owner.timerdone();
        // I'm done
    }
}
```



Madhavan Mukund

Callbacks

Programming Concepts using Java 3 / 6

सिद्धिर्भवति कर्मजा



Implementing callbacks

■ Code for Myclass

■ Timer t should know whom to notify

- Myclass m passes its identity when it creates Timer t

■ Code for Timer

- Interface Runnable indicates that Timer can run in parallel

■ Timer specific to Myclass

```
public class Myclass{
```

```
    public void f(){
```

```
        ..
```

```
        Timer t =
```

```
            new Timer(this);
```

```
            // this object
```

```
            // created t
```

```
        ...
```

```
        t.start(); // Start t
```

```
        ...
```

```
    } public void timerdone(){...}
```

```
}
```

```
public class Timer
```

```
    implements Runnable{
```

```
    // Timer can be
```

```
    // invoked in parallel
```

```
    private Myclass owner;
```

```
    public Timer(Myclass o){
```

```
        owner = o; // My creator
```

```
    }
```

```
    public void start(){
```

```
        ...
```

```
        owner.timerdone();
```

```
        // I'm done
```

```
    }
```

```
}
```



So, you have some function which is running inside myclass, and at some point, it defines a new timer and starts it. So, timer is a class which it knows about, so it creates a new timer. And what is it doing? Well, it is trying to tell this timer that this is the object that created the timer because the timer needs to know who to notify, which object to notify at the end of its execution. So, this passes this class, this object of this class when it creates a timer it will pass its own identity through the parameter, the constructor of the timer, so it starts it off. And it has a separate implementation of the function which a timer is supposed to call when it finishes. So, timer done is now provided as a function inside myclass.

So, if we look at the corresponding code in timer, it will have a constructor and this constructor will take as a parameter, the name of the object that created it. So, it keeps an owner as an instance variable and now it starts. And when start, finishes execution, it reaches the end and it calls back this owner with this function timer done. So, this is called part of the specification of myclass that it has this timer done. So, therefore, timer knows it can call it back.

So, there is a small problem with this particular implementation which is that it is created by an object of type myclass. So, I have a timer which will work only with myclass. Of course, this paradigm of calling a timer and doing a callback is general. So, we can imagine many classes

might want to do something like this. So, it would be good to have a timer class which works with multiple owner classes. So, we want a kind of a generic version of the timer.

(Refer Slide Time: 05:40)



A generic timer

- Use Java class hierarchy
- Parameter of **Timer** constructor of type **Object**
 - Compatible with all caller types

```

public class MyClass{
    public void f(){
        ..
        Timer t =
            new Timer(this);
        // this object
        // created t
        ...
        t.start(); // Start t
        ...
    }
    public void timerdone(){...}
}
        
```

```

public class Timer
    implements Runnable{
    // Timer can be
    // invoked in parallel
    private Object owner;
    public Timer(Object o){
        owner = o; // My creator
    }
    public void start(){
        ...
        ((Myclass) owner).timerdone();
        // I'm done
    }
}
        
```

Madhavan Mukund CallBack Programming Concepts using Java 4 / 6



A generic timer

- Use Java class hierarchy
- Parameter of **Timer** constructor of type **Object**
 - Compatible with all caller types
- Need to cast **owner** back to **Myclass**

```

public class MyClass{
    public void f(){
        ..
        Timer t =
            new Timer(this);
        // this object
        // created t
        ...
        t.start(); // Start t
        ...
    }
    public void timerdone(){...}
}
        
```

```

public class Timer
    implements Runnable{
    // Timer can be
    // invoked in parallel
    private Object owner;
    public Timer(Object o){
        owner = o; // My creator
    }
    public void start(){
        ...
        ((Myclass) owner).timerdone();
        // I'm done
    }
}
        
```

Madhavan Mukund CallBack Programming Concepts using Java 4 / 6

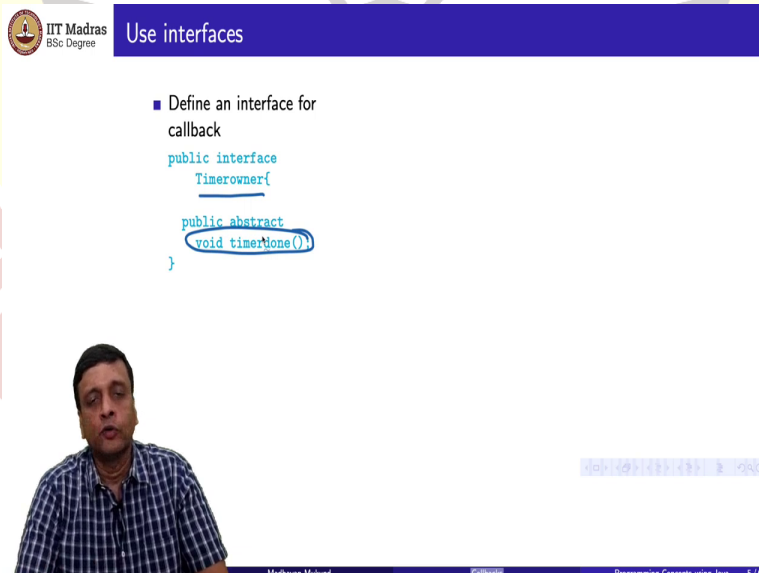
So, we know from experience that we have this Java class hierarchy, so we can make the owner of this timer class generic by allowing the owner to be of arbitrary type object. So, what we have is that the constructor now takes any object as the owner. So, the owner type is now object inside

the class, everything else is the same from myclass' point of view. So, myclass creates a timer, passes its own reference and starts it and then waits for timer data.

Now, the problem is that, in this case, the timer has to, at the end, call timer done, but all it knows about its calling class is that it is an object, and obviously, the generic object class in Java need not have this very specific function which we have created called timer done. So, we have to cast it. So, this is a problem here. We have to do this extra cast. And now, again, we are stuck, because we need to cast it back to the class set called. Earlier we were trying to avoid having a dependence in the class set called by allowing this to be an arbitrary object, but when we call it back, we need to know that this function is there. So, we need to know that the calling object has a capability and we need to invoke it. And to invoke it we have to cast it back.

So, this is, again, typical situation which is asking for an interface, because we want some capability of this class, we want the myclass has a capability to be informed through timer done. So, that is what we need to know about it. So, an interface captures this kind of limited capability. So, the natural solution to this now is to do this whole thing using interfaces.

(Refer Slide Time: 07:19)



IIT Madras
BSc Degree

Use interfaces

- Define an interface for callback

```
public interface  
    Timerowner{  
  
    public abstract  
        void timerDone()  
    }  
}
```

Madhavan Mukund

Callback

Programming Concepts using Java 5/6

Use interfaces

- Define an interface for callback

```
public interface
Timerowner{

    public abstract
    void timerdone()
}
```

- Modify **Myclass** to implement **Timerowner**

```
public class Myclass
implements Timerowner{

    public void f(){
        ..
        Timer t =
        new Timer(this);
        // this object
        // created t
        ...
        t.start(); // Start t
        ...
    }

    public void timerdone(){...}
}
```



Madhavan Mukund

Callbacks

Programming Concepts using Java 5/6

Use interfaces

- Define an interface for callback

```
public interface
Timerowner{

    public abstract
    void timerdone()
}
```

- Modify **Myclass** to implement **Timerowner**

- Modify **Timer** so that **owner** is compatible with **Timerowner**

```
public class Myclass
implements Timerowner{

    public void f(){
        ..
        Timer t =
        new Timer(this);
        // this object
        // created t
        ...
        t.start(); // Start t
        ...
    }

    public void timerdone(){...}
}
```

```
public class Timer
implements Runnable{

    // Timer can be
    // invoked in parallel
    private Timerowner owner;

    public Timer(Timerowner o){
        owner = o; // My creator
    }

    public void start(){
        ...
        owner.timerdone();
        // I'm done
    }
}
```

Madhavan Mukund

Callbacks

Programming Concepts using Java 5/6

So, we create an interface **Timerowner**. So, any class which implements **Timerowner** is guaranteed now to implement this function **timerdone()** which the timer will call back. So, this is the property that the timer needs. The owner must be capable of executing the function **timer done**. So, now we make our class that we started with **myclass** implement this interface, so implement **timer owner**. So, in particular, inside nothing changes. It still has a **timer done** function. And this **timer done** now satisfies the property that it concretizes the abstract function that the interface promises.

So, everything else is the same, except for the fact that you have kind of implemented this timer owner, nothing changes from the perspective of myclass. So, what happens in the timer now is that we have this timer owner now as the type of the owner class. So, timer owner is now the parameter which comes in and it is now assigned to this owner instance variable. But because timer owner is an interface which is guaranteed to have timer done, at the end, I can just call `owner.timerdone()` without doing any casting, because owner is compatible with the interface timer owner, and therefore, it has an implementation of timer.

So, in this way, we neatly combine the two requirements that we had that we have a kind of generic timer. This timer can be used by any class which implements this timer owner interface. And the timer need not know anything more about the class that calls it except that it implements this interface. So, I forgot to mention that there is one piece of code here, which is there in the timer definition. So, it implements an interface called runnable.

So, this is an internal Java interface that we will read, learn about later on. And this is what allows a timer to run in parallel. So, this is something which is just there to for completeness. But at the moment we have not discussed exactly what runnable means. It is not relevant to this. You just have to take it on faith that if timer implements runnable, you can actually launch this timer in parallel so that there is some meaning to this calling back.

Here we have a different situation where we call a function and then we continue to run, and then at some point later in our behavior we get called back and then we have to take some action based on that. So that is the difference between this and a conventional call. So, it is crucial that the object that we are invoking through this is running in parallel, because otherwise the callback is very simple. So, we want it to be running in parallel with the timer, so the timer can come back and somehow interrupt our execution.

(Refer Slide Time: 10:35)

- Callbacks are useful when we spawn a class in parallel
- Spawned object notifies the owner when it is done
- Can also notify some other object when done
 - `owner` in `Timer` need not be the object that created the `Timer`
- Interfaces allow this callback to be generic
 - `owner` has to have the capability to be notified



So, in general, this is a situation that happens whenever we have this kind of parallelism. We want to do many things. So, imagine, for instance, in a browser, so you have a browser, and then you click on a download button, so now this thing will start downloading. Of course, when downloading, it does not stop you from opening another tab and seeing another window, so you are reading something else.

At some point, you realize this download is taking too long, maybe it is a very large file, you did not anticipate or there is a slow connection or something, so now you can go there and you can click a stop download button. So, this stop download button will effectively notify the download process that it must stop.

So, this is again something which is happening in parallel. The browser in parallel is displaying some interesting information to you in one window and somewhere else in the background it is downloading something. So, you want to kind of go back to that process and in this case it is not automatic, it is manual, but it is triggered by some action you do. So, when you click on that button, internally, a function gets called in the download process and terminates it. So, calling back or interrupting through a function which comes back is a very essential thing you do whenever you want to do things in parallel.

And of course, the reason you want to do things in parallel is so that you can optimize the resources. I mean, no computer is kind of doing anything sequentially. Every time we use a computer we are implicitly doing lots of things because it is, for instance, the background is typically checking whether new mail has arrived for us, it is running a clock, is doing various things. So, there is always this illusion of parallelism. Even though it might be doing only one thing at a time is doing it so fast that it keeps switching back and forth and you think it is doing many things at the same time.

So, the critical capability will be what is that that spawned object must be able to go back and notify the owner. Now, in this case, the owner was the object that created the timer. But in principle, I could have an owner or a creator, in some sense, a creator, which creates a timer and tells it to notify somebody else.

So, there is no reason why the object that is passed to the timer must be the object that called it. So, in general, you can say when you are done, pass on a message to somebody else or it might say create something else. So, therefore this owner is an abstract thing. It is need not necessarily the class that called the timer. So, all we want is the timer should know that whichever object it has to notify has the capability of being notified and that is what we achieved by making that an interface. So the capability of being notified is what actually we have to convey. Who actually is notified is not a concern to us. It may be the owner, the person who created the timer or somebody else